

A I P O C K E T A G E N C Y

aipocketagency.com

[apa · kit 03]

The CLAUDE.md Template Library

Six opinionated CLAUDE.md templates — SaaS, contractor SaaS, marketing site, mobile app, API service, internal tool — that put the agent on-rails from your first commit instead of your hundredth.

KIT

03 / 05

PRICE

\$15 USD

FORMAT

10 pages

TABLE OF CONTENTS

01	Why CLAUDE.md earns its keep faster than any other file	p.2
02	What every CLAUDE.md must contain	p.3
03	How to pick the right template	p.5
04	The six templates	p.6
05	The 30-minute first-pass fill	p.9
06	Keeping CLAUDE.md current	p.10
07	What comes next	p.10

SECTION 1.

Why CLAUDE.md earns its keep faster than any other file

A good CLAUDE.md saves you eight to twelve hours of context-repetition over the first month of any project. A bad one — or no CLAUDE.md at all — costs you that and more, every month, until you write it.

The first session in a new repo with Claude Code is the same shape every time. You explain the stack. You explain the conventions. You explain what the agent should never touch. You explain where secrets live. The agent does good work for an hour. Then auto-compact fires, or you start a new session, and the next session asks the same questions because none of that context was written down anywhere durable.

CLAUDE.md is the durable place. Claude Code reads it automatically at session start. Cursor reads it. Codex respects it. Most modern coding agents — and a growing list of MCP-aware tools — pick it up without configuration. Write it once, get the answers loaded for free, every session, forever.

The trap is writing it from scratch in the first hour of every new project. CLAUDE.md is meta-work. It feels like it's not real work. You skip it. You promise to fill it in later. Later doesn't come. Six weeks in, the agent has contradicted itself across sessions four times and you finally write the CLAUDE.md you should have written on day one — except now you have to reverse-engineer your own conventions from a half-finished codebase.

This kit eliminates that friction. Six pre-built CLAUDE.md templates calibrated for the six project shapes solo operators and small teams build most. You pick the closest one. You spend thirty minutes personalizing the stack and conventions sections. The agent reads the file at session start and inherits a working agent rulebook on its very first request.

Every template ships with the same eight-section scaffold, so once you've used one, all six feel familiar. They differ in **opinions** — which Postgres library, which auth provider, which deploy target, which gotchas to call out. Those opinions are calibrated against the project shape, and they're the part you'd spend the most time on if you wrote the file alone.

WHAT THE KIT IS

Six ready-to-edit CLAUDE.md templates, plus a one-page decision tree for picking which one fits, plus the 30-minute first-pass fill guide. Drop the template into your repo root as CLAUDE.md, fill in the five personalization slots, commit, and the agent reads it on the next session start.

SECTION 2.

What every CLAUDE.md must contain

The eight sections below are universal. Every template in the kit uses the same scaffold. Once you've filled out one, the next five feel like swapping in the project-specific details.

A CLAUDE.md is not a README. The README is for humans. The CLAUDE.md is for agents. The shape that works — across every template in the kit — is short, ordered, link-heavy, and ruthlessly editable. The eight sections below are the universal scaffold.

1. Project context

What this repo is, what it isn't, who the audience is, what stage the project is at. Two paragraphs maximum. The agent uses this to decide tone — a marketing site agent talks differently than an internal-tool agent. Don't make it figure out the tone from the codebase.

2. Stack overview

The opinionated stack list with one-line rationale per choice. "React Query for server state — because we want optimistic updates by default." "Bun, not npm — because the install-test-build loop is 3x faster on this team." The rationale is the part that makes the choice durable across agents and across operators. Without rationale, every new agent suggests swapping React Query for SWR every two weeks.

3. Code conventions

Either a short list of the rules ("no `any` types," "no `console.log` in production," "no silent catches," "additive-only migrations," "direct `fetch` in server routes — never SDKs") or a one-line reference to a separate conventions doc. The Dev-Team Document Set kit ships a canonical conventions doc you can drop in alongside CLAUDE.md. If you're starting smaller, inline them here.

4. Agent rules

What the agent should always do (rebase on main first, cite SHAs not "pushed," report blockers instead of guessing) and what it should never do (force-push, skip pre-commit hooks, write secrets to files). Two short lists. This is the section that prevents the worst kinds of agent self-help.

5. Gotchas

The three to five things that trip up every operator on this stack. "Vercel sensitive env vars cannot be read back via the API — verify by exercising the code path, not by listing env vars." "Supabase migrations apply alphabetically by filename — name them with timestamp prefixes." "iOS Safari blocks popup-opened windows on user-action delay — pre-open the window synchronously." These are the things you wish someone had told you on day one. Now you tell the agent.

6. What to read first

An ordered list of files the agent should read at session start. `docs/conventions/CODING.md` , `docs/DECISIONS.md` , the most recent change log entry, the active feature inventory. The point is to give the agent a guided startup sequence so it doesn't drown in the codebase or — worse — guess at structure.

7. What to never touch

Files and directories the agent must not modify without explicit operator approval. `migrations/` , `*.lockb` , anything in `vendor/` , generated type files. The list saves you the "the agent reformatted bun.lockb and now the build is broken" Tuesday.

8. Deploy commands

The exact bash for first deploy and subsequent deploys. `bun run deploy:prod` , `supabase functions deploy <name>` , `vercel --prod` . Three commands tops. The agent uses this to either run the deploy itself (with explicit approval) or to assemble the right report when it hands the deploy back to you.

SECTION 3.

How to pick the right template

Six templates, one decision tree. Most operators land on the closest match in under a minute.

MULTI-TENANT? If yes — *SaaS app* (template 1) or *Contractor SaaS* (template 2).

PUBLIC SITE, NO *Marketing site* (template 3).

AUTH?

IOS OR ANDROID? *Mobile app* (template 4) — React Native + Expo defaults.

BACKEND-ONLY / *API service* (template 5).

API?

INTERNAL TEAM *Internal tool* (template 6) — simpler defaults, no RLS.

TOOL?

HOME SERVICES? *Contractor SaaS* (template 2) — leads → jobs → quotes → invoicing.

When in doubt between two templates, pick the one whose stack section is **closer** to what you're actually using. The conventions and agent rules are largely identical across templates — the gap is in the stack opinions. The closer the stack section, the less personalization work.

SECTION 4.

The six templates

Each template's signature — when to pick it, what stack opinions it carries, the gotchas it pre-bakes. The full file lives in the kit zip; this section is the elevator pitch.

Template 1 — SaaS app

Pick when: multi-tenant, customer-facing, billing involved. Next.js + Supabase + Stripe + Vercel.

Stack opinions baked in: React Query for server state. Bun for package management. Tailwind for styling. Row Level Security on every table. Stripe via direct REST `fetch` calls in API routes (never the SDK in server routes). Edge functions for long-running webhooks. Supabase Auth for email + OAuth, with explicit fallback to magic links for popup-blocker scenarios.

Gotchas pre-baked: Sensitive Vercel env vars are write-only via API. Stripe webhook signature verification cannot tolerate body re-reads. RLS bypass via service role key must be flagged in PR descriptions. iOS Safari popup-blocker breaks OAuth-style redirects without synchronous window pre-open. Calendly and similar webhook handlers must be built from real payloads, not synthetic samples.

Template 2 — Contractor SaaS

Pick when: home services or B2B field operations. Leads → jobs → quotes → invoicing.

Stack opinions baked in: Same Next.js + Supabase + Stripe + Vercel base, with mobile-first field-ops layout conventions, PDF quote generation via `@react-pdf/renderer`, Stripe Connect Express for contractor payouts (with the cost-acknowledgment hooks for the cross-account fee structure), and the lead-intake conventions calibrated for the contractor sales motion — Twilio webhooks for SMS lead capture, Google Maps geocoding for service-area validation, photo upload with EXIF-stripping before storage.

Gotchas pre-baked: Stripe Connect Express onboarding has a multi-step KYC flow that can be paused mid-stream; the resume URL must be stored on the contractor record. SMS opt-out compliance varies by state. Photo uploads from mobile browsers must strip EXIF before storage to avoid leaking GPS data.

Template 3 — Marketing site

Pick when: landing pages, blog, SEO. No customer auth. Next.js + Sanity (or MDX) + Vercel.

Stack opinions baked in: App Router with Server Components by default. MDX for blog posts; Sanity for content edited by non-engineers. PostHog (or Clarity) for analytics. OpenGraph image generation via `@vercel/og`. Sitemap and robots.txt generated at build time. No client-side state library — the site renders server-first.

Gotchas pre-baked: OG image generation has a hard memory limit on Vercel's free tier. Sitemap dates must be sourced from the CMS, not the file system. Social-share debuggers cache aggressively — verify with cache-busting query strings before declaring a deploy clean. Facebook in-app browser fights some redirect patterns; detect and degrade gracefully.

Template 4 — Mobile app

Pick when: iOS-first or Android-first native build. React Native + Expo + EAS Build.

Stack opinions baked in: Expo managed workflow as the default. Expo Router for navigation. NativeWind (Tailwind for RN) for styling. Sentry for crash reporting from the first commit. OTA updates via EAS Update for non-native changes. The iOS-vs-web-vs-PWA decision tree — if you can ship as PWA and the feature works in mobile Safari, do that first; only graduate to native when the feature needs a platform API the PWA can't reach.

Gotchas pre-baked: Apple Developer account approval can take days; budget for it. EAS Build credentials must be set up once, then reused; do not regenerate per build. App Store review timing is unpredictable; plan releases with a one-week buffer. PWA-to-native graduation costs roughly two weeks of work — don't pre-optimize for it before validating that the PWA isn't enough.

Template 5 — API service

Pick when: backend-only or microservice. Node + Fastify or Hono. No frontend in this repo.

Stack opinions baked in: Fastify for high-throughput services; Hono for edge-deployed services. Zod for request and response validation. OpenAPI generation from the Zod schemas at build time. Rate limiting via `@fastify/rate-limit` with per-route thresholds in a config file. Auth middleware that respects an explicit `X-Service-Token` header for service-to-service calls and a JWT bearer for user-scoped calls. Structured logging via `pino` with request IDs threaded through every log line.

Gotchas pre-baked: OpenAPI generation drifts from the Zod schemas if you cache the generator output — regenerate on every build. Rate-limit headers vary across CDNs; pick one and document the choice in the deploy section. Pino's transport-vs-direct mode has different performance implications in dev and prod; the template's dev/prod split is non-negotiable.

Template 6 — Internal tool

Pick when: single-user or small-team admin dashboard. No public surface. Often a CRM, ops tool, or internal queue UI.

Stack opinions baked in: Next.js with simpler defaults — no RLS (the data model is single-tenant), magic-link or SSO auth only, no public OG metadata. The "you don't need every

convention you'd ship in a SaaS template" caveats live in this template so you don't over-engineer an internal tool. SQLite via Turso or single-region Postgres on Supabase, depending on how many users you expect. PostHog optional. Vercel deploy is fine; the tool doesn't need a multi-region story.

Gotchas pre-baked: "Internal" can drift to "external" when a contractor joins the team. Plan the auth boundary as if external access might happen. Magic-link auth has a different rate-limit story than passworded auth. Sensitive operations (delete-record, change-permissions) need explicit confirmation flows even with one user — agents will happily run a destructive action when asked.

SECTION 5.

The 30-minute first-pass fill

Every template has five personalization slots. Filling them takes thirty minutes. The agent reads the result on the next session.

The five slots are the same across all six templates:

1. **Project name and one-line description.** §1 of the template. Two sentences, max.
2. **Stack overrides.** §2 lists opinionated defaults. If you're using something different — `pnpm` instead of `bun`, `trpc` instead of `REST`, `Drizzle` instead of `Kysely` — replace the default and add a one-line rationale. If the default is fine, leave it. The default is the recommendation.
3. **Code conventions reference.** §3 either links to a separate conventions doc (recommended if you have the Dev-Team Document Set kit) or lists the rules inline. Pick one.
4. **Gotchas — add yours.** §5 ships with the stack-specific gotchas. Add the project-specific ones as you hit them. The first three are often: a specific API quirk you've already learned, a deploy gotcha specific to your account, a teammate's preference that's worth respecting.
5. **Deploy commands.** §8 — the exact bash for first deploy and subsequent deploys. If you don't have a deploy yet, leave the placeholder; come back when you deploy.

The other three sections (Agent Rules, What to Read First, What to Never Touch) are universal and ship pre-filled. They almost never need customization on the first pass.

THE 30-MINUTE TEST

After your first pass, open a fresh Claude Code session in the repo, ask: "What is this project, what's the stack, and what's the first file you'd read?" If the agent answers all three from CLAUDE.md alone, you're done. If it asks follow-ups, the answer is in §1 or §2 of your template — go put it there.

SECTION 6.

Keeping CLAUDE.md current

A stale CLAUDE.md is worse than no CLAUDE.md. The agent reads it as truth. Stale truth is a bug factory.

The rule that works: **every decision log entry is a CLAUDE.md trigger**. If you log a decision that changes the stack, conventions, agent rules, or deploy commands, you update CLAUDE.md in the same commit. The decision log is the trigger; CLAUDE.md is the destination.

The other rule that works: **monthly audit**. Once a month, you read CLAUDE.md top to bottom. You ask, “is each line still true?” The lines that aren’t, you fix. The audit takes ten minutes. Skip it for three months and CLAUDE.md silently becomes the file the agent uses to make confidently wrong suggestions.

If you’re using the Dev-Team Document Set kit alongside this one, the cascade staleness convention (`stale-audit.sh`) flags CLAUDE.md as a downstream dependent of every convention doc and ADR. Run the audit script after any upstream change; if CLAUDE.md shows up in the report, that’s your update trigger.

SECTION 7.

What comes next

CLAUDE.md is the on-ramp. The Dev-Team Document Set is the artifact stack. The AI Pocket Agency community is the live system that keeps them current as Claude, Cursor, and Codex all ship breaking changes.

This kit pairs cleanly with the Dev-Team Document Set kit. Together they're \$25 — six CLAUDE.md templates plus eleven document templates plus three operational conventions, dropped into your repo as a complete agent rulebook on day one.

The system around the kits — patterns for evolving these templates as the agent landscape ships breaking changes — is the AI Pocket Agency community at aipocketagency.com. Founding 50 is \$47/month locked for life. New CLAUDE.md template variants ship there before anywhere else. The brain dashboard wires all the docs together so you can see them as live signal instead of grep-only history.

Drop a template in. Fill the five slots. Watch the next session start with the agent already understanding your project.

— Chase